Name (Last, First): Answer Key

This problem is about `39k_virtual_memory`: building the kernel's page table and turning on translation. There is 1 problem, in 3 parts, on 4 pages. Answer each part in the space below it, and show your work.

1. **Turning on virtual memory (39k).** In 39k, `kvmmake` builds the kernel's page table, and `kvminithart` hands it to the MMU. These are the addresses involved:

```
pub const PGSIZE: usize = 4096;
pub const KERNBASE: usize = 0x8000_0000;     // RAM starts here (kernel code, data, stacks)
pub const PHYSTOP: usize = 0x8800_0000;      // one byte past the end of RAM (128 MB)
pub const UART0: usize = 0x1000_0000;        // the serial port (printing)
pub const TEST_FINISHER: usize = 0x10_0000;  // the device that powers QEMU off
```

   (a) The register `satp` tells the MMU which page table to use. Its bits are packed like this, and `make_satp` builds the value:

```
//  satp bits 63..60: MODE  (8 = Sv39, 3-level page tables; 0 = translation off)
//       bits 59..44: ASID  (0 for us)
//       bits 43..0 : PPN   (physical page number of the ROOT page table)
pub const SATP_SV39: usize = 8 << 60;

pub fn make_satp(root: *mut Pte) -> usize {
    SATP_SV39 | ((root as usize) >> 12)
}
```

   `kvmmake`'s first `kalloc()` returns the root page table at `0x87fff000`. What does `make_satp(root)` return? Give it in hex, and say what the MODE and PPN fields hold. Why is the root address shifted right by 12?

   0x87fff000 >> 12 = 0x87fff, and 8 << 60 = 0x8000_0000_0000_0000, so

   make_satp(root) = 0x8000_0000_0008_7fff

   MODE = 8: Sv39, the 3-level scheme. ASID = 0. PPN = 0x87fff, the root table's physical page number.

   A page table fills exactly one page-aligned page, so the low 12 bits of its address are always 0: satp stores only the page number (address / 4096).

(b) Here is how `kvmmake` fills the table. Each page table is one page holding 512 entries; `px(level, va)` picks the entry for `va` at each level (2 = root, 0 = last):

```
pub const PTE_V: usize = 1 << 0;  pub const PTE_R: usize = 1 << 1;  // valid, readable
pub const PTE_W: usize = 1 << 2;  pub const PTE_X: usize = 1 << 3;  // writable, executable
pub struct Pte(pub usize);                              // one 8-byte page-table entry
impl Pte {
    pub const fn new(pa: usize, flags: usize) -> Pte { Pte(((pa >> 12) << 10) | flags) }
    pub const fn pa(self) -> usize { (self.0 >> 10) << 12 }    // the page it points to
    pub const fn is_valid(self) -> bool { self.0 & PTE_V != 0 }
}
fn pgrounddown(a: usize) -> usize { a & !(PGSIZE - 1) }  // start of a's page
const fn px(level: usize, va: usize) -> usize {      // 9 bits of va per level
    (va >> (12 + level * 9)) & 0x1ff
}
pub unsafe fn walk(mut table: *mut Pte, va: usize, alloc: bool) -> *mut Pte {
    let mut level = 2;
    while level > 0 {
        let pte = table.add(px(level, va));
        if (*pte).is_valid() {
            table = (*pte).pa() as *mut Pte;          // follow to the next-level table
        } else {
            if !alloc { return ptr::null_mut(); }
            let page = kalloc::kalloc();               // allocate a new, empty table
            if page.is_null() { return ptr::null_mut(); }
            ptr::write_bytes(page, 0, PGSIZE);
            *pte = Pte::new(page as usize, PTE_V);
            table = page as *mut Pte;
        }
        level -= 1;
    }
    table.add(px(0, va))                               // the leaf entry for va
}
pub unsafe fn mappages(table: *mut Pte, va: usize, size: usize, pa: usize,
                       perm: usize) -> Result<(), ()> {
    let mut a = pgrounddown(va);
    let last = pgrounddown(va + size - 1);
    let mut pa = pa;
    loop {                                             // one leaf entry per page
        let pte = walk(table, a, true);
        if pte.is_null() { return Err(()); }
        *pte = Pte::new(pa, perm | PTE_V);
        if a == last { break; }
        a += PGSIZE;
        pa += PGSIZE;
    }
    Ok(())
}
pub unsafe fn kvmmake() -> *mut Pte {
    let root = kalloc::kalloc() as *mut Pte;           // the root table
    if root.is_null() { return ptr::null_mut(); }
    ptr::write_bytes(root as *mut u8, 0, PGSIZE);
    // identity maps: va == pa
    if mappages(root, UART0, PGSIZE, UART0, PTE_R | PTE_W).is_err()
        { return ptr::null_mut(); }
    if mappages(root, TEST_FINISHER, PGSIZE, TEST_FINISHER, PTE_R | PTE_W).is_err()
        { return ptr::null_mut(); }
    if mappages(root, KERNBASE, PHYSTOP - KERNBASE, KERNBASE,
                PTE_R | PTE_W | PTE_X).is_err()
        { return ptr::null_mut(); }
    root
}
```

(i) Using the code above, fill in the indexes for the two device pages. (ii) `kvmmake` maps `TEST_FINISHER` right after `UART0`. While mapping `TEST_FINISHER`, does `walk` allocate a new level-1 table? A new level-0 table? Use your indexes from (i) and the `walk` code to explain.

va	px(2, va)	px(1, va)	px(0, va)
UART0 = 0x1000_0000	0	128	0
TEST_FINISHER = 0x10_0000	0	0	256

(i) UART0 >> 12 = 0x10000: px(0) = 0x10000 & 0x1ff = 0; >> 21 = 0x80 = 128; >> 30 = 0. TEST_FINISHER >> 12 = 0x100 = 256: px(0) = 256, px(1) = px(2) = 0.

(ii) walk allocates a table only when the entry it needs is still empty (not valid).

Level 1: no new table. px(2) = 0 for both devices, and mapping UART0 already filled root entry 0, so walk just follows it to the existing level-1 table.

Level 0: yes, one new table. In that level-1 table, UART0 used entry px(1) = 128, but TEST_FINISHER needs entry 0, which is still empty → walk calls kalloc() for a new level-0 table.

So mapping TEST_FINISHER allocates exactly 1 page-table page. The leaf entry itself allocates nothing: it points to the device's own page, 0x10_0000.

(c) `kvminithart` switches translation on:

```
pub unsafe fn kvminithart(root: *mut Pte) {
    let satp = make_satp(root);
    asm!("csrw satp, {}", in(reg) satp);   // from here on, the MMU uses this table
    asm!("sfence.vma zero, zero");         // flush any old cached translations (TLB)
}
```

Once the kernel runs in supervisor mode (from 43k on), *every* address it uses is a virtual address, and the MMU translates it with this table. (In 39k it still runs in machine mode, which `satp` does not translate.) In Rust, the MMU does this:

```
fn translate(root: *mut Pte, va: usize) -> Option<usize> {
    let pte = walk(root, va, false);       // follow px(2), px(1) down to the leaf entry
    if pte.is_null() || !(*pte).is_valid() {
        return None;                       // an empty entry on the way: PAGE FAULT
    }
    Some((*pte).pa() + (va & 0xfff))       // the leaf's page + the offset (low 12 bits)
}
```

Remember that `kvmmake` mapped exactly three regions: the UART page (`0x1000_0000`), the `TEST_FINISHER` page (`0x10_0000`), and all of RAM (`0x8000_0000` up to `0x8800_0000`). Nothing else is mapped, so translating any other address is a page fault.

Translate each address with `kvmmake`'s table. The first row is done for you. Then answer: compare each result with its `va`. Why will the kernel keep running normally once its addresses are translated?

va	px(2)	px(1)	px(0)	offset	result (pa, or page fault)
0x8000_1234 (kernel code)	2	0	1	0x234	pa = 0x8000_1234
0x1000_0005 (in the UART page)	0	128	0	0x5	pa = 0x1000_0005
0x4000_0000	1	0	0	0x0	page fault

Example, row 1: `0x8000_1234 >> 12 = 0x80001`, so `px(0) = 0x80001 & 0x1ff = 1`, `px(1) = (0x80001 >> 9) & 0x1ff = 0`, `px(2) = 0x80001 >> 18 = 2`; offset = `0x234`. Root entry 2, then level-1 entry 0, then level-0 entry 1 are all valid (the RAM mapping), and the leaf points to page `0x8000_1000`, so `pa = 0x8000_1000 + 0x234 = 0x8000_1234`.

UART row: same indexes as (b); the UART page is mapped, so pa = 0x1000_0000 + 5 = 0x1000_0005.

0x4000_0000 row: it is in none of the three mapped regions → page fault. In the table: px(2) = 1, but kvmmake only filled root entry 0 (the devices) and root entry 2 (RAM), so root entry 1 is empty and walk returns null at the first step.

Every mapped address translates to itself (identity map, va == pa), so the PC, the stack and every pointer the kernel already holds still reach the same bytes: turning translation on changes nothing the kernel can see.